

Validating Hex Editors on macOS and Linux

Hex editors are essential forensic tools that allow investigators to view and manipulate raw binary data at the byte level. Before using any such tool in an investigation, its validity and reliability must be confirmed. Without proper validation, an examiner cannot be certain that a hex editor is accurately reading or displaying file data. This paper examines four hex editors, two for macOS and two for Linux, and describes a validation methodology for each.

Hex Editors for macOS

The two hex editors selected for macOS are Hex Fiend and 010 Editor. Hex Fiend is a free, open-source editor built for macOS that handles large files and supports binary file comparison. 010 Editor is a commercial tool offering scripting, binary templates, and thorough documentation, which simplifies understanding its intended behavior and designing validation tests.

Hex Editors for Linux

For Linux, the selected tools are GHex and Bless Hex Editor. GHex is an open-source editor maintained under the GNOME project with a clean interface and wide distribution support. Bless offers tabbed editing and a plugin system. Both are open source, meaning their source code can be reviewed alongside documentation to verify expected behavior, a significant advantage during validation.

Validation Methodology

Validation confirms that a tool performs as expected and produces accurate, repeatable results. The general approach for each hex editor follows a common framework: establish known inputs, define expected outputs, run the tool, and compare results. This process must be thoroughly documented so that another examiner can reproduce it and obtain identical findings.

The first step is to create known test files in various formats, plain text, images, and binary executables, with pre-computed MD5 and SHA-256 hash values recorded before testing begins. These hashes establish a baseline confirming no unintended modifications occurred. A simple text file with a known ASCII string, for instance, can be manually documented in hexadecimal and then opened in each editor to verify that the displayed byte values match expectations.

For Hex Fiend and GHex, validation focuses on read accuracy and display integrity. The tester opens each known test file, verifies that the hexadecimal and ASCII panels show correct byte values, and confirms that file offsets align with expected positions. Search functions are tested by locating known byte sequences and confirming the tool identifies their correct locations within the file.

For 010 Editor and Bless, additional steps address their advanced features. 010 Editor's binary templates would be tested by applying known templates to matching file types and comparing parsed output against documented format specifications. Bless's plugin functionality would be tested by running a plugin against a known file and confirming consistent output across runs. In all cases, files should be re-hashed after editing to confirm only the intended bytes were changed.

Reproducibility is a core forensic requirement. Each test should be run at least three times and, ideally, repeated on a second machine running the same OS version. Results must be identical across all runs. Any discrepancies must be investigated and documented before the tool is approved for casework.

Ease of Validation Based on Documentation

All four tools have publicly available documentation, though depth varies. 010 Editor provides the most comprehensive coverage, including scripting and template language specifications, making it the easiest to validate because expected behaviors are precisely defined. Hex Fiend's documentation is lighter but sufficient for its simpler feature set. GHex and Bless have moderate documentation, but their open-source code can be reviewed directly to verify behavior at a deeper level when needed.

Conclusion

Validating hex editors requires a disciplined approach built on known inputs, documented expected outputs, and repeatable testing. Hex Fiend and 010 Editor for macOS, along with GHex and Bless for Linux, each present a clear validation path given the availability of their documentation. By hashing test files before and after use, verifying displayed byte values, and testing all major features against documented specifications, a forensic examiner can establish a defensible record confirming that each tool performs accurately before it is used in an actual investigation.

Retrieving Already Deleted Files

```
kali@kali: ~  
Session Actions Edit View Help  
  
(kali@kali)-[~]  
$ sudo fdisk -l  
Disk /dev/sdb: 4 MiB, 4194304 bytes, 8192 sectors  
Disk model: VBOX HARDDISK  
Units: sectors of 1 * 512 = 512 bytes  
Sector size (logical/physical): 512 bytes / 512 bytes  
I/O size (minimum/optimal): 512 bytes / 512 bytes  
Disklabel type: gpt  
Disk identifier: 8454CE98-8548-4079-BF92-45972FB2A026  
  
Device      Start      End Sectors Size Type  
/dev/sdb1   2048      8158    6111    3M Microsoft basic data  
  
Disk /dev/sda: 80.09 GiB, 86000000000 bytes, 167968750 sectors  
Disk model: VBOX HARDDISK  
Units: sectors of 1 * 512 = 512 bytes  
Sector size (logical/physical): 512 bytes / 512 bytes  
I/O size (minimum/optimal): 512 bytes / 512 bytes  
Disklabel type: dos  
Disk identifier: 0x3a69c69b  
  
Device      Boot Start      End  Sectors  Size Id Type  
/dev/sda1   *      2048 167968749 167966702 80.1G 83 Linux
```

```
kali@kali: ~  
Session Actions Edit View Help  
  
(kali@kali)-[~]  
$ sudo apt-get install foremost  
[sudo] password for kali:  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
foremost is already the newest version (1.5.7-11+b2).  
Solving dependencies... Done  
The following packages were automatically installed and are no longer required:  
  amass-common base58 binutils-mingw-w64-base binutils-mingw-w64-i686  
  binutils-mingw-w64-x86-64 debugedit dnsmasq ettercap-common ettercap-graphical figlet finger  
  firmware-ti-connectivity gcc-mingw-w64-base gcc-mingw-w64-i686-win32  
  gcc-mingw-w64-i686-win32-runtime gcc-mingw-w64-x86-64-win32  
  gcc-mingw-w64-x86-64-win32-runtime gir1.2-girepository-2.0 gir1.2-nm-1.0 gir1.2-vte-2.91  
  greenbone-feed-sync gvmd-common imagemagick imagemagick-7.q16 libann0 libapache2-mod-php  
  libarmadillo14 libaudio2 libavfilter10 libavformat61 libblosc2-6 libbluray2  
  libbson-1.0-0t64 libcdt5 libcgraph6 libconfig-inifiles-perl libdisplay-info2 libdlt2  
  libfsverity0 libgav1-1 libgdal36 libgdal-common libgdal22 libgeos3.13.1  
  libgirepository-1.0-1 libgmpxx4ldbl libgpgme11t64 libgpgmepp6t64 libgvm22t64 libhdf4-0-alt  
  libhiredis1.1.0 libical3t64 libinstpatch-1.0-2 libio-pty-perl libipc-run-perl  
  libjs-jquery-ui libjs-underscore liblab-gamut1 liblua5.1-2 liblua5.1-common liblz4-1  
  libmjpegutils-2.1-0t64 libmongoc-1.0-0t64 libmpeg2encpp-2.1-0t64 libmplex2-2.1-0t64  
  libmupdf25.1 libnet1 libobjc-14-dev libogdi4.1 libpaho-mqtt1.3 libplacebo349  
  libpocketsphinx3 libportmidi0 libpostproc58 libqt5ct-common1.8 libradare2-5.0.0t64  
  libradcli4 librpm-build10 librpm-sign10 librubberband2 libsegment1 libsigsegv2 libsmb2-6  
  libsnmp40t64 libsoup-2.4-1 libsoup2.4-common libspatialite3t64 libsqlcipher1 libswscale8
```

```

(kali㉿kali)-[~]
$ sudo foremost -i /dev/sda -o /home/kali/Desktop/Test
Processing: /dev/sda
|**WMV err num_header_objs=14978577 headerSize=14963569906419540224
*WMV err num_header_objs=1735815968 headerSize=4994590205195273249
WMV err num_header_objs=-1 headerSize=18446744073709486080
*WMV err num_header_objs=-3368832 headerSize=18432275604491245752
*foundat=vk_swiftshader_icd.json*VJ*I*O*/M,/K-*S*RP2*3*3P*QP*tv*r2*****
K2@

***"e*****i%*)*Ez** ***&***
foundat=xdg-mime*=kwf****+P*(*)YI*{*Di[I*ul*cc*)*x-*$*m*ww
foundat=xdg-settings*=*W?*
Ö*8~*****v      *h*{d{*;*;3*$*~*<

```

